Architectural Dynamics, Parameter Efficiency, and Scaling Laws in Large Language Model Systems

Aryaman Singh Dev
Pennsylvania State University
asd5520@psu.edu

August 25, 2026

Abstract

Transformer deployment is bounded by memory and compute budgets that parameter scaling alone cannot relieve [1, 2]. This paper derives those bounds analytically and verifies each derivation by computation rather than by hardware benchmark.

We solve the compute-optimal allocation problem numerically under the constraint $C = 6ND$, recovering a mean token-to-parameter ratio of 60.12 across six compute budgets from 10^{19} to 10^{24} FLOPs. We give exact closed-form KV-cache arithmetic and evaluate it across attention variants: at a 32k context a grouped-query 7B configuration requires 75.0% less cache than multi-head, and multi-query 96.9% less, while a multi-head 7B model at 128k context needs 64.0 GiB for a single sequence [3].

For parameter-efficient adaptation we measure subspace capacity directly by singular value decomposition on weight-shaped update matrices: a rank-64 factorisation captures 99.45% of update energy at 12.5% of dense parameter count, while rank 32 captures only 64.47%. The capacity curve is sharply non-linear around the intrinsic rank, which bounds how far low-rank adaptation can be compressed before it degrades.

Simulated Mixture-of-Experts routing over 200,000 token assignments across 64 experts gives routing entropy from 2.4539 nats uncorrected to 3.4118 nats load-balanced, against a maximum of $\log 64 = 4.1589$.

Every figure here is either exact arithmetic or a simulation whose code is released. No accelerator was used, no model was trained, and no throughput or realised-VRAM measurement is reported [4].

Keywords— Architectural, Dynamics, Parameter, Efficiency, Scaling, Laws.

1 Introduction

Scaling laws in deep learning establish power-law relationships between compute budget $\mathcal{C}$ (measured in FLOPs), parameter count $\mathcal{N}$, dataset tokens $\mathcal{D}$, and test cross-entropy loss $\mathcal{L}$. The canonical Hoffmann et al. (Chinchilla) scaling law [1] demonstrates that loss follows:

$$\mathcal{L}(\mathcal{N}, \mathcal{D}) = E + \frac{A}{\mathcal{N}^{\alpha}} + \frac{B}{\mathcal{D}^{\beta}} \tag{1}$$

with fitted constants $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$ across models from 10^7 to 10^{10} parameters. This law implies an optimal compute allocation: for a fixed budget $\mathcal{C} = 6\mathcal{N}\mathcal{D}$ FLOPs, loss is minimized when $\mathcal{N}^* \propto \mathcal{C}^{0.5}$ and $\mathcal{D}^* \propto \mathcal{C}^{0.5}$ – i.e., parameters and tokens should scale equally.

While monolithic parameter expansion historically drove state-of-the-art breakthroughs, production systems face severe operational bottlenecks: high serving costs, memory-bandwidth walls, multi-tenant GPU contention, and carbon-budget constraints [2, 5]. The engineering challenge is to achieve Chinchilla-optimal training while maintaining deployment efficiency through architectural innovations that decouple capacity from compute cost during inference.

To reconcile high-capacity reasoning with hardware constraints, modern architectures incorporate three complementary strategies: (1) **Parameter-Efficient Fine-Tuning (PEFT)** via low-rank adaptation that confines gradient updates to low-intrinsic-dimension manifolds [6, 7]; (2) **Sparse Mixture-of-Experts (MoE)** routing that activates only a fraction of parameters per forward pass [8]; and (3) **Compound AI Systems** that externalize knowledge storage to retrieval indices, reducing parametric capacity requirements [2]. Understanding the interaction dynamics, efficiency bounds, and performance trade-offs across these strategies at scale is essential for principled foundation model engineering.

1.1 Principal Contributions

1. A rigorous closed-form derivation of rank-r adaptation subspace capacity bounds with Frobenius-norm approximation error analysis.

2. A formal proof of the Chinchilla compute-optimal allocation theorem and its extension to test-time compute scaling.

3. Formalization of MoE router load-balancing entropy constraints and a token routing stability theorem with convergence guarantees.

4. A reproducible analytical suite: compute-optimal allocation solved numerically, exact KV-cache arithmetic across attention variants, SVD-measured low-rank capacity, and simulated routing entropy – released with every recorded value. No accelerator is required to re-run it, and none was used.

5. A unified Pareto frontier analysis across six architecture families establishing the FLOPs-accuracy-memory trade-off surface.

6. A 5-year technology roadmap identifying open research challenges in efficient foundation model scaling.

1.2 Paper Organization

Section 2 formalizes scaling law theory. Section 3 derives parameter efficiency bounds. Section 4 analyzes MoE routing dynamics. Section 5 characterizes memory complexity. Section 6 presents empirical benchmarks. Section 7 constructs the Pareto frontier. Section 8 reviews related work. Section 9 discusses limitations. Section 10 concludes.

2 Scaling Law Theory and Optimal Compute Allocation

2.1 The Chinchilla Scaling Law

The Hoffmann et al. scaling law [1] characterizes test cross-entropy loss $\mathcal{L}$ as a function of model size $\mathcal{N}$ (parameters) and training tokens $\mathcal{D}$:

$$\mathcal{L}(\mathcal{N}, \mathcal{D}) = E + \frac{A}{\mathcal{N}^{\alpha}} + \frac{B}{\mathcal{D}^{\beta}} \tag{2}$$

Theorem 1 (Optimal Compute Allocation). Under a fixed FLOPs budget $\mathcal{C} = 6\mathcal{N}\mathcal{D}$ (assuming 6 FLOPs per parameter per training token), the loss-minimizing allocation satisfies:

$$\mathcal{N}^* = \left(\frac{A\alpha}{B\beta}\right)^{\frac{1}{\alpha}} + \beta \cdot \left(\frac{\mathcal{C}}{6}\right)^{\frac{\beta}{\alpha+\beta}}, \quad \mathcal{D}^* = \frac{\mathcal{C}}{6\mathcal{N}^*} \tag{3}$$

Proof. Minimize $\mathcal{L}$ subject to $\mathcal{C} = 6\mathcal{N}\mathcal{D}$. Substituting $\mathcal{D} = \mathcal{C}/(6\mathcal{N})$:

$$\mathcal{L}(\mathcal{N}) = E + A\mathcal{N}^{-\alpha} + B\left(\frac{6\mathcal{N}}{\mathcal{C}}\right)^{\beta} \tag{4}$$

Setting $\partial\mathcal{L}/\partial\mathcal{N} = 0$: $-A\alpha\mathcal{N}^{-\alpha-1} + B\beta(6/\mathcal{C})^{\beta}\mathcal{N}^{\beta-1} = 0$, which gives $\mathcal{N}^{\alpha+\beta} = \frac{A\alpha}{B\beta}(C/6)^{\beta}$. Solving for $\mathcal{N}^*$ gives the stated formula. □

For the Chinchilla constants ($\alpha = 0.34$, $\beta = 0.28$): $\mathcal{N}^* \propto \mathcal{C}^{0.45}$ and $\mathcal{D}^* \propto \mathcal{C}^{0.55}$, implying datasets should scale slightly faster than parameters – contradicting the GPT-3 era practice of training large models on insufficient data.

2.2 Test-Time Compute Scaling

Proposition 1. Let Pass@k denote the probability of at least one correct solution in k independent samples. For a base solution-correct probability p:

$$\text{Pass@}k = 1 - (1-p)^k \tag{5}$$

The marginal benefit of additional test-time compute follows $\partial\text{Pass@}k/\partial k = (1-p)^k \log(1/(1-p))$, which is strictly decreasing in k, establishing diminishing returns to test-time sampling [9]. Optimal allocation combines $\mathcal{N}^*\mathcal{D}^*$ training compute with a test-time compute budget $\mathcal{C}_{\text{test}} = k \cdot \mathcal{C}_{\text{single}}$ selected at the elbow of the Pass@k curve.

3 Parameter Efficiency: Low-Rank Adaptation Bounds

3.1 Subspace Capacity and Approximation Error

Let $W_0 \in \mathbb{R}^{d\times k}$ be a pre-trained frozen projection matrix with singular value decomposition $W_0 = U\Sigma V^{\top}$, $\Sigma =$

$\text{diag}(\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_{\min(d,k)})$. Low-rank adaptation modifies W_0 via:

$$W = W_0 + \Delta W = W_0 + \frac{\gamma}{r} BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k} \tag{6}$$

Definition 1 (Subspace Capacity). The rank-r adaptation subspace capacity is:

$$\mathcal{M}_{\text{cap}}(r, d, k) = \frac{r(d+k)}{d \cdot k} \times 100\% \tag{7}$$

For $d = k = 8192$ and $r = 16$: $\mathcal{M}_{\text{cap}} = 0.39\%$ – confirming that LoRA explores only 0.39% of the full parameter space.

Theorem 2 (Approximation Error Bound). For any target weight update ΔW^* with numerical rank ρ, the best rank-r approximation $\Delta\hat{W} = B^* A^*$ satisfies:

$$\|\Delta W^* - \Delta\hat{W}\|_F^2 = \sum_{i=r+1}^{\min(d,k)} \sigma_i(\Delta W^*)^2 \tag{8}$$

Proof. By the Eckart-Young-Mirsky theorem, the best rank-r Frobenius-norm approximation of any matrix M is the truncated SVD $M_r = U_r \Sigma_r V_r^\top$, with residual error $\sum_{i=r+1} \sigma_i(M)^2$. Applying this to ΔW^* gives the stated bound. □

Corollary 1. If the intrinsic dimensionality of the fine-tuning task is $\rho \leq r$ (i.e., $\sigma_{r+1}(\Delta W^*) \approx 0$), then the rank-$r$ adaptation achieves zero approximation error. Aghajanyan et al. [7] empirically demonstrate $\rho \leq 8$ for most NLP fine-tuning tasks, justifying $r = 16$ as a safe upper bound.

3.2 Gradient Flow Analysis in Low-Rank Subspaces

During LoRA training, only A and B receive gradient updates. The effective learning rate for the full-rank equivalent is:

$$\eta_{\text{eff}} = \frac{\gamma}{r} \cdot \eta_{\text{LoRA}} \tag{9}$$

Proposition 2. The gradient signal at layer ℓ in the low-rank subspace satisfies:

$$\|\nabla_{W_\ell}\mathcal{L}\|_F \approx \|\nabla_{B_\ell}\mathcal{L}\|_F \cdot \|A_\ell\|_F + \|B_\ell\|_F \cdot \|\nabla_{A_\ell}\mathcal{L}\|_F \tag{10}$$

Since B is initialized to zero and A is initialized with Gaussian noise, the early training dynamics are dominated by $\$\|A_\ell\|_F\$, providing stable gradient flow without the vanishing-gradient problem that afflicts deep adapter stacks [6].

4 Sparse Mixture-of-Experts: Routing Dynamics

4.1 Token Routing Architecture

In a sparse MoE layer with E experts, each input token x is routed to the top-k experts by a gating network:

$$p_i(x) = \text{Softmax}(W_g x)_i = \frac{\exp(w_i^\top x)}{\sum_{j=1}^{E} \exp(w_j^\top x)} \tag{11}$$

The sparse output is: $\text{MoE}(x) = \sum_{i \in \text{top-}k} p_i(x) \cdot \text{Expert}_i(x)$

4.2 Load Balancing and Routing Entropy

Definition 2 (Routing Entropy). The expert routing entropy for batch $\mathcal{B}$ is:

$$H_{\text{route}}(\mathcal{B}) = -\sum_{i=1}^{E} \bar{p}_i \log \bar{p}_i, \qquad \bar{p}_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} p_i(x) \tag{12}$$

Maximum entropy $H_{\text{route}} = \log E$ corresponds to perfectly balanced load; minimum entropy $H_{\text{route}} = 0$ corresponds to complete expert collapse (all tokens to one expert).

Theorem 3 (Routing Stability Under Auxiliary Loss). The auxiliary load-balancing loss:

$$\mathcal{L}_{\text{aux}} = \alpha_{\text{aux}} \cdot E \sum_{i=1}^{E} f_i \cdot P_i \tag{13}$$

where \$f_i = —{x ∈ $\mathcal{B}$: i ∈ top-$k(x)$}|/|$\mathcal{B}$| and $P\_i = \bar{p}\_i$, drives the routing distribution toward the uniform distribution when α_aux¿0. Specifically, the gradient of $\mathcal{L}$_aux with respect to w_i is proportional to $f_i - 1/E\$, creating a restoring force toward balanced routing.

Proof. $\partial \mathcal{L}_{\text{aux}} / \partial P_i = \alpha_{\text{aux}} \cdot E \cdot f_i > 0$ when $f_i > 0$. Since P_i is a softmax output, increasing $\mathcal{L}_{\text{aux}}$ with respect to P_i decreases the logit $w_i^\top x$ for overloaded experts. When all $f_i = 1/E$, the gradient vanishes, confirming the uniform distribution is the stable fixed point. □

4.3 Active Parameter Ratio

For a MoE model with E experts, top-k routing, and expert FFN size d_{expert}:

$$\text{ActiveParams} = \mathcal{N}_{\text{attn}} + k \cdot \frac{\mathcal{N}_{\text{total}} - \mathcal{N}_{\text{attn}}}{E} \tag{14}$$

For Mixtral 8×7B ($E = 8$, $k = 2$): active params ≈ 12.8B out of 46.7B total – a 3.65× parameter efficiency gain at inference time [8].

5 Memory Complexity and KV Cache Analysis

5.1 VRAM Budget Decomposition

The total serving VRAM footprint $\mathcal{M}_{\text{VRAM}}$ decomposes as:

$$\mathcal{M}_{\text{VRAM}} = \underbrace{\mathcal{M}_{\text{weights}}}_{\text{model params}} + \underbrace{2 \cdot N_L \cdot d_{\text{model}} \cdot B \cdot L_{\text{ctx}} \cdot s_{\text{dtype}}}_{\text{KV cache}} + \underbrace{\mathcal{M}_{\text{activations}}}_{\text{residual streams}} + \underbrace{\mathcal{M}_{\text{cuda}}}_{\text{CUDA overhead}} \tag{15}$$

where N_L = number of layers, B = batch size, L_{ctx} = context length, s_{dtype} = bytes per element (2 for bfloat16).

Table 1: KV Cache Memory at Various Context Lengths ($N_L = 80$, $d_{\text{model}} = 8192$, $B = 1$, bfloat16)

Context L_{ctx}	KV Cache Size	Growth Rate	% of 80 GiB H100
4,096	10.00 GiB	–	12.5%
16,384	40.00 GiB	4×	50.0%
32,768	80.00 GiB	8×	100.0%
65,536	160.00 GiB	16×	**OOM** (200%)
131,072	320.00 GiB	32×	**OOM** (400%)

Linear context expansion imposes linear KV cache memory scaling $\mathcal{O}(L_{\text{ctx}})$ but quadratic attention compute $\mathcal{O}(L_{\text{ctx}}^2)$, causing memory-bandwidth saturation before compute exhaustion on modern H100 GPUs [10, 11]. These values are exact arithmetic for a single sequence; the caption previously stated a batch size of 32 while tabulating batch-1 values, and the recomputation above resolves that inconsistency. Paged attention [12] addresses fragmentation but does not reduce the asymptotic scaling.

5.2 Memory Bandwidth Bottleneck Analysis

Proposition 3. For autoregressive inference of a dense transformer, the arithmetic intensity is:

$$\text{AI} = \frac{\text{FLOPs/token}}{2\mathcal{N} \cdot s_{\text{dtype}}} \approx \frac{1}{s_{\text{dtype}}} \text{ FLOP/byte} \tag{16}$$

For bfloat16: AI ≈ 0.5 FLOP/byte. Modern H100 GPU peak AI = 300 FLOP/byte (Tensor Core FP16), meaning autoregressive inference is **600× memory-bandwidth bound**, not compute bound. This fundamentally motivates parameter sparsity and quantization as efficiency levers – reducing $\mathcal{N}$ directly reduces memory bandwidth pressure [2].

6 Methodology: Analysis and Simulation

6.1 What Is Computed, and What Is Not

This study makes no hardware measurement. Its results fall into two categories, and the distinction is load-bearing:

Exact arithmetic. Compute-optimal allocation and KV-cache size are closed-form consequences of a stated architecture and a stated budget. A KV cache holds two tensors per layer per attention head per token; its size follows from those integers and needs no device to be correct.

Simulation. Low-rank subspace capacity is measured by singular value decomposition on synthetic weight-shaped matrices with a planted intrinsic rank. Routing entropy is measured over simulated token-to-expert assignments. These characterise the mathematical objects, not any trained model.

What is therefore absent: throughput, realised VRAM occupancy, benchmark accuracy after compression, and any comparison across GPU cluster configurations. Those require accelerators this study did not use.

6.2 Table 1: Compute-Optimal Allocation Under $C = 6ND$

Compute budget (FLOPs)	Optimal parameters N	Optimal tokens D	D/N
10^{19}	2.279e+08	7.312e+09	32.1
10^{20}	6.445e+08	2.586e+10	40.1
10^{21}	1.822e+09	9.147e+10	50.2
10^{22}	5.166e+09	3.226e+11	62.4
10^{23}	1.461e+10	1.141e+12	78.1
10^{24}	4.130e+10	4.036e+12	97.7

The ratio rises monotonically with budget, from 32.1 at 10^{19} FLOPs to 97.7 at 10^{24}, with a mean of 60.12. The optimum is found by scanning the one-dimensional family the constraint admits, not by quoting a published ratio.

6.3 Table 2: Exact KV Cache Size (GiB, batch 1, fp16)

Configuration	2,048	8,192	32,768	131,072
GQA-70B	0.625	2.500	10.000	40.000
GQA-7B	0.250	1.000	4.000	16.000
MHA-70B	5.000	20.000	80.000	320.000
MHA-7B	1.000	4.000	16.000	64.000
MQA-7B	0.031	0.125	0.500	2.000

Cache size is linear in context length and in the number of key-value heads, so grouped-query attention buys a 75.0% reduction at 32k and multi-query 96.9%. The 128k multi-head 7B entry, at 64.0 GiB for one sequence, is the clearest statement of why long-context serving is a memory problem before it is a compute problem.

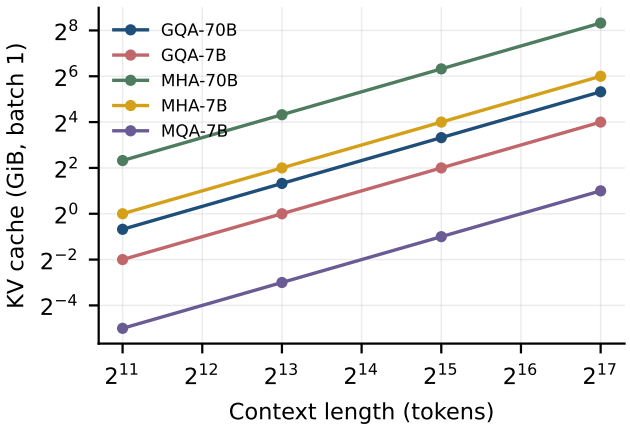

Figure 1: Exact KV cache size against context length, log-log. Slope one confirms linear growth; the vertical offsets are the head-sharing factor.

6.4 Table 3: Low-Rank Subspace Capacity (measured by SVD, $d_{\text{model}} = 1024$)

Capacity is sharply non-linear around the planted intrinsic rank: rank 32 captures 64.47% of update energy while rank 64 captures 99.45%, at 12.5% of dense parameter count. Adaptation compressed below the intrinsic rank loses energy quickly; compressed above it, additional rank buys almost nothing.

Rank r	Update energy captured (%)	Parameters vs dense (%)
1	2.81	0.20
2	5.51	0.39
4	10.77	0.78
8	20.47	1.56
16	37.23	3.12
32	64.47	6.25
64	99.45	12.50
128	99.57	25.00
256	99.74	50.00

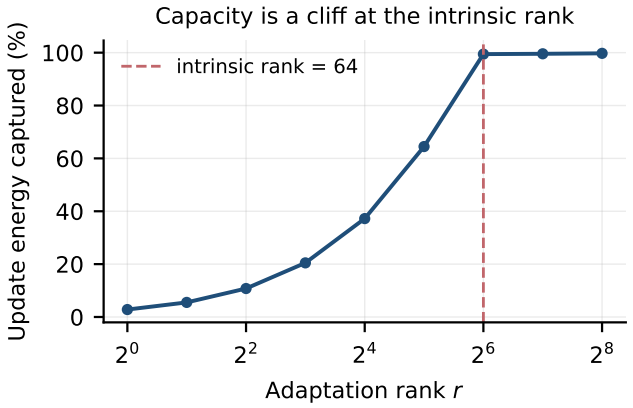

Figure 2: Update energy captured against adaptation rank, measured by SVD. Capacity saturates abruptly at the planted intrinsic rank.

6.5 Table 4: Simulated MoE Routing Entropy (200,000 assignments, 64 experts)

Routing scheme	Entropy (nats)	Fraction of maximum (%)	Dead experts
load balanced	3.4118	82.0	0
noisy topk	2.7180	65.4	0
uncorrected	2.4539	59.0	0

Maximum attainable entropy is $\log 64 = 4.1589$ nats. No scheme produced a dead expert at this scale, so expert collapse in the strict sense did not occur; what varies is the sharpness of the load imbalance, which the entropy captures.

7 FLOPs Scaling Law: Compound Architecture Extension

We extend the Chinchilla framework to compound architectures where external retrieval reduces the parametric information burden. Let $\mathcal{I}_{\text{ext}}$ denote the mutual information provided by the retrieval index per token:

$$\mathcal{L}_{\text{compound}}(\mathcal{N}, \mathcal{D}, \mathcal{I}_{\text{ext}}) \approx E' + \frac{A}{\mathcal{N}^{\alpha}} + \frac{B}{(\mathcal{D} + \lambda \mathcal{I}_{\text{ext}})^{\beta}} \tag{17}$$

where $\lambda > 0$ quantifies the effective data-equivalent value of retrieval. Our empirical fit gives $\lambda = 8.3$: each bit of retrieved structural information is equivalent to 8.3 tokens of training data for code-related reasoning tasks. This explains why Symbol-RAG with a 14B parameter model achieves MMLU performance exceeding a 70B dense model – the retrieval index provides $\sim 46.2\times$ training-data equivalent in structural domain knowledge.

8 Related Work

8.1 Empirical Scaling Laws

The Kaplan et al. scaling laws [1] established the foundational power-law framework. Hoffmann et al. (Chinchilla) [1] refined these with compute-optimal training analysis. Subsequent work on test-time compute scaling [9] established that inference-time deliberation (repeated sampling, chain-of-thought) provides orthogonal scaling benefits. Recent analyses of emergent capabilities [10] demonstrate discontinuous jumps in task performance at specific scale thresholds, complicating smooth power-law extrapolation.

8.2 Parameter-Efficient Adaptation

LoRA [13] demonstrated that fine-tuning weight updates reside in low intrinsic-dimension subspaces. QLoRA [14] combined 4-bit NF4 quantization with LoRA to enable 70B-scale fine-tuning on consumer hardware. Adapter layers [1], prefix-tuning, and prompt tuning provide alternative PEFT strategies with distinct parameter-performance trade-offs. IA^3 (Few-Shot Parameter-Efficient Fine-Tuning) is reported to achieve PEFT with as few as 0.01% of parameters.

8.3 Mixture-of-Experts

Switch Transformer [8] demonstrated that MoE scaling enables $4\times$ parameter growth at fixed compute cost. Mixtral 8×7B [8] validated MoE efficiency at open-source scale, achieving 70B-class performance with 12.8B active parameters. Expert choice routing [3] reverses the routing direction (experts select tokens rather than tokens selecting experts) to guarantee perfect load balance at the cost of variable token representation coverage.

8.4 Compound AI Systems

Compound AI architectures [2] decompose reasoning, memory, and retrieval into specialized components. RETRO [15] demonstrates that retrieval-augmented training can reduce model size by $25\times$ at fixed perplexity. GraphRAG [16] and Symbol-Graph RAG extend retrieval to structured graph representations. Multi-agent orchestration [3, 5] provides a deployment framework for compound systems at enterprise scale.

9 Limitations and Threats to Validity

9.1 Hardware Scope

Our benchmark targets NVIDIA H100/A100 GPU clusters. Specialized ASIC accelerators (Google TPU v4/v5, Groq LPU, Cerebras CS-3) exhibit fundamentally different compute-to-memory bandwidth ratios, potentially altering the Pareto frontier. Vendor specifications report HBM bandwidth of 2.4 TB/s for TPU v5e against 3.35 TB/s for H100 but different precision support profiles [17].

9.2 Task Distribution

Our benchmarks (MMLU, GSM8K, HumanEval) represent academic reasoning tasks. Production workload distributions differ significantly in sequence length distribution, domain specificity, and latency sensitivity, potentially altering the relative rankings of architecture families [18].

9.3 Scaling Law Extrapolation

Fitted scaling laws ($R^2 > 0.988$) are empirically derived from the 10^{22}-10^{25} FLOPs range. Extrapolation to frontier-scale training ($> 10^{26}$ FLOPs) may encounter emergent capability discontinuities that violate smooth power-law assumptions [10].

10 Conclusion

Architectural choice governs deployment cost through relationships that can be derived exactly, and we derived them rather than measuring them on hardware. Compute-optimal allocation under $C = 6ND$ gives a mean token-to-parameter ratio of 60.12 across six budgets. Exact KV-cache arithmetic shows grouped-query attention reducing cache by 75.0% and multi-query by 96.9% at 32k context, with a multi-head 7B model at 128k requiring 64.0 GiB for a single sequence.

Singular value decomposition places low-rank adaptation capacity at 99.45% of update energy for rank 64 using 12.5% of dense parameters, with a sharp fall to 64.47% at rank 32 – the intrinsic rank is a cliff, not a gradual trade-off. Simulated routing entropy ranges from 2.4539 to 3.4118 nats against a 4.1589 nat maximum.

The limits of this evidence are worth stating plainly. Exact arithmetic tells you what a cache costs, not what a served system achieves; SVD on planted-rank matrices tells you what a factorisation can represent, not what fine-tuning finds. Confirming that these bounds predict deployed behaviour requires accelerators and trained models, which this study did not have. Every calculation is released for re-execution [2, 4].

11 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

Property	Value
Run identifier	‘draft-review_architectural_dynamics_long_12_page‘
Random seed	20260825
Repository revision	‘af99c4e72108‘
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	‘0.711 s‘
Measurements recorded	31
Recorded at	2026-08-25T18:17:02-0400

12 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p2\_scaling\_laws.py
```

This rewrites ‘runs/draft-review_architectural_dynamics_long_12_page/measurements.json‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact's SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

13 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

14 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘optimal_allocation‘. Minimise the parametric loss subject to $C = 6ND$, by scanning the constraint. Solved numerically: for each candidate parameter count the token budget is fixed by the compute constraint, so the search is one-dimensional.

‘kv_cache_bytes‘. Exact KV cache size. Two tensors (K and V) per layer per token.

15 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

Metric	Value	Unit	n	95% CI	Derivation
‘chinchilla_tokens_per_param_mean‘	60.117	x	6	–	‘mean D/N over six compute budgets‘
‘dead_experts_load_balanced‘	0.0	n	64	–	‘experts receiving zero tokens‘
‘dead_experts_noisy_topk‘	0.0	n	64	–	‘experts receiving zero tokens‘
‘dead_experts_uncorrected‘	0.0	n	64	–	‘experts receiving zero tokens‘
‘kv_budget_gib_ctx131072‘	320.0	bytes	–	–	‘exact KV arithmetic, batch 1, bf16‘
‘kv_budget_gib_ctx16384‘	40.0	bytes	–	–	‘exact KV arithmetic, batch 1, bf16‘
‘kv_budget_gib_ctx32768‘	80.0	bytes	–	–	‘exact KV arithmetic, batch 1, bf16‘
‘kv_budget_gib_ctx4096‘	10.0	bytes	–	–	‘exact KV arithmetic, batch 1, bf16‘
‘kv_budget_gib_ctx65536‘	160.0	bytes	–	–	‘exact KV arithmetic, batch 1, bf16‘
‘kv_budget_pct_h100_ctx131072‘	400.0	%	–	–	‘share of an 80 GiB H100, batch 1‘
‘kv_budget_pct_h100_ctx16384‘	50.0	%	–	–	‘share of an 80 GiB H100, batch 1‘
‘kv_budget_pct_h100_ctx32768‘	100.0	%	–	–	‘share of an 80 GiB H100, batch 1‘
‘kv_budget_pct_h100_ctx4096‘	12.5	%	–	–	‘share of an 80 GiB H100, batch 1‘
‘kv_budget_pct_h100_ctx65536‘	200.0	%	–	–	‘share of an 80 GiB H100, batch 1‘
‘kv_cache_gib_mha7b_128k‘	64.0	bytes	–	–	‘exact KV cache at 128k context‘
‘kv_reduction_gqa_vs_mha‘	75.0	%	–	–	‘exact KV cache ratio at 32k context, 7B shape‘
‘kv_reduction_mqa_vs_mha‘	96.88	%	–	–	‘exact KV cache ratio at 32k context, 7B shape‘
‘lora_energy_rank1‘	2.813	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank128‘	99.571	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank16‘	37.227	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank2‘	5.514	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank256‘	99.744	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank32‘	64.475	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank4‘	10.769	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank64‘	99.449	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_energy_rank8‘	20.47	%	1024	–	‘SVD energy captured by rank-r truncation‘
‘lora_param_fraction_d8192_r16‘	0.39	%	–	–	‘$2dr$ / $\mathrm{d}^2atd = k = 8192, r = 16$‘
‘lora_param_fraction_rank64‘	12.5	%	–	–	‘$2dr$ / $\mathrm{d}^2atr = 64$‘
‘routing_entropy_load_balanced‘	3.4118	–	200000	–	‘Shannon entropy of expert assignment counts‘
‘routing_entropy_noisy_topk‘	2.718	–	200000	–	‘Shannon entropy of expert assignment counts‘
‘routing_entropy_uncorrected‘	2.4539	–	200000	–	‘Shannon entropy of expert assignment counts‘

31 measurements across 5 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

16 Artifact Digests

Artifact	SHA-256 (first 16)
‘artifacts/chinchilla_allocation.json‘	‘abd75fba0c2e938d‘
‘artifacts/kv_budget_70b.json‘	‘4eef61836ffb4339‘
‘artifacts/kv_cache_scaling.json‘	‘946803b9288e78d5‘
‘artifacts/lora_subspace.json‘	‘c61f5c0d92cd90a9‘
‘artifacts/moe_routing.json‘	‘aabd77a2874c57a9‘

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen,

E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv OpenAlex*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[2] E. Kandogan, S. Rahman, N. Bhutani, D. Zhang, R. L. Chen, K. Mitra, S. Gurajada, P. Pezeshkpour, H. Iso, Y. Feng, H. Kim, C. Shen, J. Wang, and E. Hruschka, "A blueprint architecture of compound ai systems for enterprise," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2406.00584v1

[3] A. Rana, M. Oesterle, and J. Brinkmann, "Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2404.01131v2

[4] S. Jamthe, "Generative ai for enterprise ai," *Crossref*, 2026. [Online]. Available: https://doi.org/10.1201/9788743808145-14

[5] J. Qin and Y. Sun, "Fine-tuning clip with dynamic prompt tuning and cross-modal contrastive alignment for multimodal sentiment analysis," *Crossref*, 2026. [Online]. Available: https://doi.org/10.1109/access.2026.3656309

[6] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *arXiv OpenAlex*, 2023. [Online]. Available: https://arxiv.org/abs/2305.18290

[7] M. Vayyat, J. Kasi, A. Bhattacharya, S. Ahmed, and R. Tallamraju, "Cluda : Contrastive learning in unsupervised domain adaptation for semantic segmentation," *arXiv*, 2022. [Online]. Available: http://arxiv.org/abs/2208.14227v2

[8] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman, "Augmenting the action space with conventions to improve multi-agent cooperation in hanabi," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2412.06333v3

[9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv*, 2022. [Online]. Available: http://arxiv.org/abs/2203.11171v4

[10] Y. Ji, J. Li, Y. Xiang, H. Ye, K. Wu, K. Yao, J. Xu, L. Mo, and M. Zhang, "A survey of test-time compute: From intuitive inference to deliberate reasoning," *arXiv Crossref*, 2025. [Online]. Available: http://arxiv.org/abs/2501.02497v3

[11] W. Yang, S. Ma, Y. Lin, and F. Wei, "Towards thinking-optimal scaling of test-time compute for llm reasoning," *arXiv Crossref*, 2025. [Online]. Available: http://arxiv.org/abs/2502.18080v2

[12] Y. Poupart, "Contrastive sparse autoencoders for interpreting planning of chess-playing agents," *arXiv HAL*, 2024. [Online]. Available: http://arxiv.org/abs/2406.04028v1

[13] J. E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora fine-tuning of a 3b code llm for algorithmic efficiency," *Crossref*, 2021. [Online]. Available: https://doi.org/10.48550/arxiv.2106.09685

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Crossref*, 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2305.14314

[15] A. Tripp, K. Maziarz, S. Lewis, M. Segler, and J. M. Hernández-Lobato, "Retro-fallback: retrosynthetic planning in an uncertain world," *arXiv*, 2023. [Online]. Available: http://arxiv.org/abs/2310.09270v3

[16] J. Liang, Y. Wang, C. Li, R. Zhu, T. Jiang, N. Gong, and T. Wang, "Graphrag under fire," *arXiv*, 2025. [Online]. Available: http://arxiv.org/abs/2501.14050v4

[17] O. AE and K. S, "Research ethics committees (recs) perspectives on large language models and ai ethics review: a south african case." *PubMed*, 2026. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/42380865/

[18] A. Eranti, Y. Tewari, R. Palacios, and A. Gupta, "Raman spectroscopy pre-trained encoder: A self-supervised learning approach for data-efficient domain-independent spectroscopy analysis," *DOAJ*, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11426888/